コミュニティーシツテム要求仕様書

Software Requirements Specification (SRS)

文書情報

項目	内容
プロジェクト名	Springベースのコミュニティーシステム
ドキュメントバージョン	1.0
作成日	2026年3月26日
作成者	朴潤正(Park YunJeong)
ステータス	ドラフト

1. プロジェクト概要

1.1 プロジェクト背景

入試前に個人でSpringを勉強するためにBoard（掲示板）を作成したいと思います。
CRUD可能な形態で会員認証、ページング、コメント、検索機能、付加機能の実装します。

1.2 プロジェクト目的

- 入社前の課題と技術勉強プロジェクトとして活用
- Sptingプレームワークを活用したWebアプリケーション開発
- Spring Securityを利用した暗号化
- CRUD ： 作成(CREATE)、参照(READ)、更新(UPDATE)、削除(DELETE)

1.3 開発スコープ

- バックエンド: Spring Framework(MVC), MyBatis, MySQL
- フロントエンド: JSP/JSTL, HTML5, CSS3, JavaScript
- デプロイ: GitHub Pages(要求仕様書)、クラウドサーバーAWS(EC2, S3)

1.4 開発スケジュール

- 総期間: 二週間「2026.03.26.～2026.04.09.」
- Phase 1 (1週目): システム設計およびデータベース構想
- Phase 2 (2週目): 付加機能開発・テスト

2. システム目標

2.1 機能目標

1. ユーザー認証「Spring Security 暗号化」
1. CRUD： 記事の作成・参照・更新・削除
1. CRUD： コメント作成・参照・更新・削除
1. 検索機能
1. いいね「非同期での部分更新」
1. ブックマーク

2.2 非機能目標

項目	目標
セキュリティ	Spring Securityベースの認証/認可
パフォーマンス	ページロード2秒以内
可用性	99% 以上
ユーザビリティ	直感的なUI，レスポンシブデザイン
保守性	MVCパターン，コードの文書化
拡張性	REST API構造

3. ユーザー定義

3.1 ユーザータイプ

3.1.1 非会員「ゲスト」

- **説明**: 会員登録していない訪問者
- **権限**:
  - 記事一覧の参照(READ)
  - 記事詳細の参照(READ)
  - 検索機能の使用
  - 会員登録ページへのアクセス

3.1.2 一般会員「ユーザー」

- **説明**: メール認証を完了した会員
- **権限**:
  - 非会員の全権限
  - 記事の作成(CREATE)/修正(UPDATE)/削除(UPDATE/soft_delete)（本人のみ）
  - コメントの作成(CREATE)/修正(UPDATE)/削除(UPDATE/soft_delete)（本人のみ）
  - いいね，ブックマーク
  - プロフィール管理

4. 機能要件

4.1 会員管理

FR-001: 会員登録

- **優先度**: 高
- **説明**: ユーザーがメールアドレスを通じて会員登録を行います。
- **入力**:
  - メールアドレス: 必須, バリデーション
  - パスワード: 必須, 8文字以上, 英数字組み合わせ
  - パスワード確認: 必須, 一致確認
  - ニックネーム 必須, 2-10文字
- **処理プロセス**:
  1. メールアドレス重複確認
  1. ニックネーム重複確認
  1. パスワードのSpring Security 暗号化
  1. usersテーブルに保存
  1. デフォルトロール(USER)付与
- **出力**:
  - 成功時はログインページへ遷移
  - 失敗時はエラーメッセージ
- **イクセプション処理**:
  - メールアドレス重複時: 「既に使用されているメールアドレスです。」
  - ニックネーム重複時: 「既に使用されているニックネームです。」

FR-002: ログイン

- **優先度**: 高
- **説明**: メールアドレスとパスワードでログインします。
- **入力**:
  - メールアドレス
  - メールアドレス
- **処理プロセス:**
  1. Spring Securityによる認証処理
  1. セッション生成
  1. ユーザー権限の確認
- **出力**:
  - 成功時はメインページへ遷移
  - 失敗時: 「メールアドレスまたはパスワードが正しくありません。」
- **非機能要件**:
  - ログイン5回失敗時、5分間アカウントロッ

FR-002: ログアウト

- **優先度**: 中
- **説明**: 現在のセッションを終了し、ログアウトします。
- **処理**: セッション無効化
- **出力**: メインページへ遷移

FR-004: プロフィール参照

- **優先度**: 中
- **説明**: ユーザー自身のプロフィール情報を参照する
- **出力**: メールアドレス，ニックネーム，加入日，投稿数

FR-005: プロフィール修正

- **優先度**: 中
- **入力**: ニックネーム，プロフィール画像（任意
- **権限**: 本人限定
- **処理**: usersテーブルの更新

FR-006: パスワード変更

- **優先度**: 中
- **入力**: 現在のパスワード，新しいパスワード，確認用パスワード
- **処理プロセス:**
  1. 現在のパスワードの確認
  1. 新しいパスワードのBCrypt暗号化
  1. 更新

4.2 記事管理

FR-101: 記事作成

- **優先度**: 高
- **説明**: ログインユーザーが新しい記事を作成します。
- **入力**:
  - タイトル: 日本語, 必須, 最大200文字
  - 本文: 日本語, 必須, 最大10,000文字
  - タグ: 任意, カンマ区切り
  - 画像: 任意, 最大5MB, jpg/png
- **処理プロセス:**
  1. 入力データのバリデーション
  1. 画像アップロード処理
  1. postsテーブルに保存
  1. タグがある場合 post_tags 保存
- **出力**:作成完了後、記事詳細ページへ遷移
- **権限**: ログインユーザー

FR-102: 記事一覧参照

- **優先度**: 高
- **説明**: 記事一覧をページングして表示します。
- **入力**:
  - ページ番号 (デフォルト: 1)
  - ページサイズ (デフォルト: 10)
  - ソート基準 (最新順/閲覧数順/いいね順)
- **出力**:
  - 記事タイトル
  - 投稿者ニックネーム
  - 投稿日
  - 閲覧数
  - いいね数
  - コメント数
  - ページング情報
- **権限**: 全ユーザー

FR-103: 記事詳細参照

- **優先度**: 高
- **説明**: 記事の全内容を参照します。
- **処理プロセス:**
  1. 閲覧数の増加（Cookieベースの重複防止，24時間）
  1. 記事データの取得
  1. コメント一覧の取得
  1. いいね済み確認（ログインユーザー）
- **出力**:
  - 記事の全内容
  - 投稿者情報
  - コメント一覧
  - 修正/削除ボタン（権限がある場合）
- **権限**: 全ユーザー

FR-104: 記事修正

- **優先度**: 中
- **説明**: 既存の記事を修正します。
- **入力**: FR-101と同じです。
- **処理:** postsテーブルの更新，updated_atの更新
- **権限**: 投稿者

FR-105: 記事削除

- **優先度**: 中
- **説明**: 記事を削除する
- **処理:** 論理削除（is_deleted = 1へ変更）
- **権限:** 投稿者
- **確認**: 「本当に削除しますか？」ポップアップ

4.3 コメント管理

FR-201: コメント作成

- **優先度**: 中
- **説明**: 記事にコメントを投稿します。
- **入力**: コメント内容（必須，最大500文字）
- **処理プロセス:**
  1. 入力バリデーション
  1. commentsテーブルに保存
  1. 記事のコメント数を増加
- **出力**: リロードなしでコメント追加「Ajax」
- **権限**: ログインユーザー

FR-202: コメント修正

- **優先度**: 低
- **権限**: 投稿者または管理者
- **処理:** commentsテーブルの更新

FR-203: コメント削除

- **優先度**: 低
- **権限**: (投稿者
- **処理:** Soft Delete(論理削除)

4.4 検索機能

FR-301: 統合検索

- **優先度**: 中
- **説明**: タイトル，本文，投稿者で検索します。
- **入力**:
  - 検索キーワード（最小2文字）
  - 検索タイプ（タイトル/本文/投稿者/全体）
- **処理:** LIKE検索，ページング適用
- **出力**: 検索結果一覧
- **権限**: 全ユーザー

FR-302: タグ別に検索

- **優先度**: 低
- **入力**: タグ名
- **出力**: 該当タグの記事一覧

4.5 コミュニケーション機能

FR-401: いいね

- **優先度**: 中
- **説明**: 記事に「いいね」を追加/取り消します。
- **処理プロセス:**
  1. 重複チェック（1ユーザーにつき1回のみ）
  1. likesテーブルへの追加/削除
  1. 記事のいいね数の増加/減少
- **出力**: リアルタイムのいいね数更新「Ajax」
- **権限**: ログインユーザー

FR-402: ブックマーク

- **優先度:** 低
- **説明**: 後で読む記事をブックマークします。
- **処理:** bookmarksテーブルに保存
- **権限**: ログインユーザー

5. 非機能要件

5.1 Security

NFR-001: 認証および認可

- Spring Securityを使用したフォーム認証
- ールベースアクセス制御 (RBAC)
- セッションタイムアウト: 30分

NFR-002: パスワードセキュリティ

- Cryptハッシュアルゴリズム使用)
- Salt自動生成
- 最小8文字, 英数字組み合わせ強制

NFR-003: CSRF対策

- Spring Security CSRFトークン使用
- 全POST/PUT/DELETEリクエストのトークン検証

5.2 Performance

NFR-101: 応答時間

- ページロード: 2秒以内
- API応答: 1秒以内
- 画像ロード: 2秒以内

NFR-102: ファイルサイズ制限

- 画像アップロード: 最大5MB
- 記事本文: 最大10,000文字

NFR-103: ページング

- 一覧参照時デフォルト10件表示
- 最大100件まで1ページに表示可能

5.3 Usability

NFR-201: ユーザーフィードバック

- ロード中のスピナー表示
- 成功/失敗メッセージをToast形式で表示
- フォームバリデーションのリアルタイム表示

NFR-202: エラー処理

- エラーメッセージは明確かつ親切に

5.4 Maintainability

NFR-301: コード構造

- MVCパターン遵守
- レイヤー分離「Controller, Service, Repository」
- パッケージ構造の明確化

NFR-302: ドキュメント化

- Java 日本語 コメント作成
- README.md 詳細作成

NFR-303: コード品質

- 変数名, メソッド名を明確に作成
- マジックナンバー禁止（定数化）
- ハードコーディング最小化（properties 活用）

NFR-304: バージョン管理

- Git使用
- 意味のあるコミットメッセージ作成
- ブランチ戦略: main, develop, feature, fix

5.5 Scalability

NFR-401: API 設計

- RESTful API 原則遵守
- JSON 形式応答

NFR-402: データベース

- 外部キー関係明確化
- 拡張可能なスキーマ設計

6. 制約事項

6.1 技術的制約

- Java 11
- Spring Framework 5.3.31
- MySQL 8.0.42

6.2 環境的制約

- 開発環境: Windows 11, STS3(Spring Tool Suite)
- デプロイ環境: 無料クラウド「AWS Free Tier」
- ポートフォリオサイト: GitHub Pages

6.3 日程制約

- 全体開発期間: 2週
- 週あたり投資時間: 約22時間(週末を含む)
- 最終締切日: 2025年4月10日

6.4 コスト制約

- 予算: 完全無料
- 無料ホスティングのみ使用
- 無料APIのみ活用

7. 優先度定義

Phase 1: コア機能（1週目）

機能 ID	機能名	優先度
FR-001	会員登録	高
FR-002	ログイン	高
FR-003	ログアウト	中
FR-101	記事作成	高
FR-102	記事一覧参照	高
FR-103	記事詳細参照	高
FR-104	記事修正	中
FR-105	記事削除	中

Phase 2: 付加機能（2週目）

機能 ID	機能名	優先度
FR-201	コメント作成	中
FR-203	コメント修正	低
FR-204	コメント削除	低
FR-301	統合検索	中
FR-302	タグ別に検索	低
FR-401	いいね	中
FR-402	ブックマーク	低
FR-004	プロフィール参照	中
FR-005	プロフィール修正	中

8. データ要件(簡易)

8.1 主要エンティティ(簡易)

users（会員）

- user_id (PK)
- email (UNIQUE)
- password (暗号化)
- nickname (UNIQUE)
- point (DEFAULT 0)
- role (USER/ADMIN)
- created_at
- is_active

posts（記事）

- post_id (PK)
- user_id (FK → users)
- title
- content
- views
- likes
- is_deleted
- created_at
- updated_at

comments（コメント）

- comment_id (PK)
- post_id (FK → posts)
- user_id (FK → users)
- content
- created_at
- is_deleted

tags（タグ）

- tag_id (PK)
- name (UNIQUE)

post_tags（記事−タグ関係）

- post_id (FK → posts)
- tag_id (FK → tags)
- PRIMARY KEY (post_id, tag_id)

likes（いいね）

- like_id (PK)
- post_id (FK → posts)
- user_id (FK → users)
- UNIQUE (post_id, user_id)

bookmarks（ブックマーク）

- bookmark_id (PK)
- post_id (FK → posts)
- user_id (FK → users)
- created_at
- UNIQUE (post_id, user_id)

リレーショナルDB説明:

- users : posts = 1 : N (ユーザーは複数記事作成可能)
- posts : comments = 1 : N (1記事に複数コメント可能)
- posts : tags = N : M (記事とタグは多対多関係, post_tags中間テーブル)
- users : bookmarks = 1 : N (ユーザーは複数記事をブックマーク可能)
- posts : bookmarks = 1 : N (1記事は複数ユーザーからブックマーク可能)

※ ユーザーと記事は実質的に多対多の関係となり、bookmarksが中間テーブルの役割を果たします。

9. インターフェース要件

9.1 ユーザーインターフェース

主要画面

1. メインページ（記事一覧）
1. ログイン/会員登録ページ
1. 記事詳細ページ
1. 記事作成/修正ページ
1. プロフィールページ
1. 検索結果ページ

10. テスト要件

10.1 統合テスト

- Spring Test 使用
- 主要APIエンドポイントテスト

10.2 ユーザー受け入れテスト

- 会員登録から記事作成までの全体フローテスト
- 追加予定: 管理者機能